Project - Low Level Design

on

Construction Content Generator

Course Name:Gen AI

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	SNEHA GUPTA	EN22CS306051
2	NAMAN SUNDRIYAL	EN22CS304042
3	AKASH SINGH	EN22CS303009
4	YASH DHIMOLE	EN22CS304066
5	HARSH NANDWAL	EN22CS304027
6	SHUBH RATHOD	EN22CS304060

Group Name: Group 12D11

Project Number: GAI-38

Industry Mentor Name:

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2025-26

Introduction.

1.1. Scope of the document.

1.2. Intended Audience

1.3. System overview.

2. System Design.

2.1. Application Design

2.2. Process Flow.

2.3. Information Flow.

2.4. Components Design

2.5. Key Design Considerations

2.6. API Catalogue.

3. References

1. Introduction

1.1 Scope of the document

This document presents the **Low Level Design (LLD)** of the **Construction Content Generator** system. It explains the internal structure of the application, its modules, data flow, processing logic, key design decisions, and function-level interfaces.

The system is designed to generate professional construction documents such as **Daily Site Reports, Weekly Progress Reports, Safety Reports, and Material Summaries** from structured user inputs. It uses **Python** as the orchestration layer, a **Vector Database** for semantic retrieval of construction knowledge, and an **LLM API** for content generation.

This document covers:

- module-wise design
- processing steps
- data movement between modules
- component responsibilities
- important design considerations
- API/function catalogue for the system

1.2 Intended Audience

This document is intended for:

- project guides and evaluators
- developers implementing the system
- team members working on backend logic
- students preparing for viva or technical review
- future contributors who may extend the system

It is useful for understanding how the system works internally and how each module contributes to the overall report-generation workflow.

1.3 System overview

The **Construction Content Generator** is a domain-specific AI-based content generation system that automates construction documentation.

Construction professionals often prepare documents manually, which is repetitive and time-consuming. This system reduces that effort by:

1. accepting structured construction-related input
2. retrieving relevant construction templates and terminology from a Vector DB
3. combining the retrieved knowledge with user input
4. generating a structured professional report using an LLM API

The system supports document types such as:

- Daily Site Report
- Weekly Progress Report
- Safety Report
- Material Summary

The overall approach follows a **Retrieval-Augmented Generation (RAG)** pipeline, where retrieval improves relevance and generation improves readability and professionalism.

2. System Design

2.1 Application Design

The application is designed as a **modular Python pipeline**. Each stage of the workflow is separated into a dedicated module to improve maintainability, testing, and scalability.

Main modules

1. **Input Handler Module**
   Captures structured report inputs from the user.

2. **Preprocessing Module**
   Cleans, validates, and normalizes input values.

3. **Query Builder Module**
   Builds a semantic search query from input data.

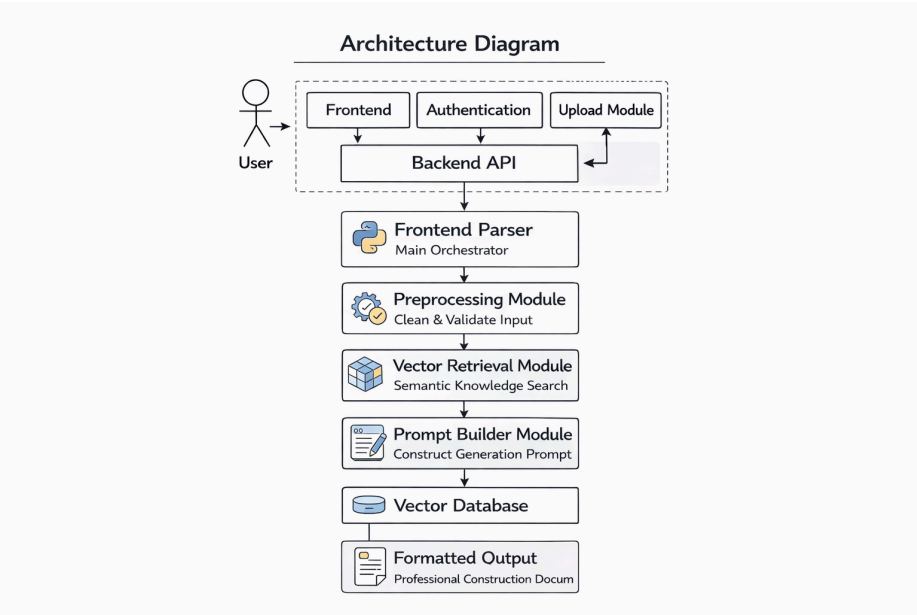
4. **Vector Retrieval Module**
   Retrieves relevant construction context from the Vector DB.

5. **Prompt Builder Module**
   Combines cleaned input and retrieved context into a prompt.

6. **LLM Integration Module**
   Sends the prompt to the LLM API and receives the output.

7. **Output Formatter Module**
   Formats the generated text into a readable construction report.

8. **Main Controller Module**
   Coordinates all modules and executes the complete pipeline.



2.2 Process Flow

The process flow of the system is as follows:

1. The user provides construction-related details such as project name, date, work completed, safety notes, and issues.

2. The system collects the data through the Input Handler.

3. The Preprocessing Module cleans the data and replaces missing values with "Not provided".

4. The Query Builder generates a semantic search query using key input fields.

5. The Vector Retrieval Module searches the Vector DB and returns relevant construction templates, terminology, and safety guidance.

6. The Prompt Builder combines:

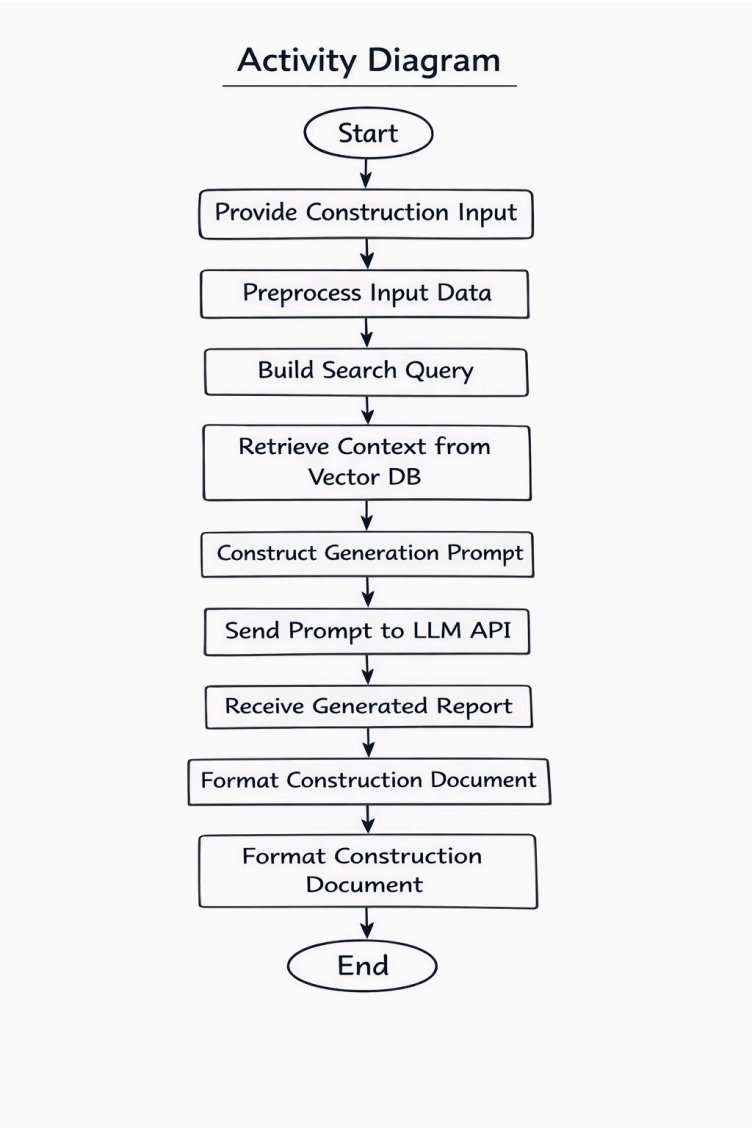
    - cleaned user input
    - retrieved context
    - formatting rules
    - professional tone instructions

7. The LLM Integration Module sends the prompt to the LLM API.

8. The LLM API returns the generated construction document.

9. The Output Formatter organizes the text into a readable final output.

10. The final report is displayed or stored.



Process summary

Input → Preprocess → Retrieve → Build Prompt → Generate → Format Output

2.3 Information Flow

The information flow describes how data moves inside the system.

Input data flow

User input is provided as structured fields:

- document_type

- project_name
- location
- date
- topic
- weather
- work_completed
- work_in_progress
- manpower
- materials
- equipment
- quality_checks
- safety
- issues
- next_plan
- remarks

Internal flow

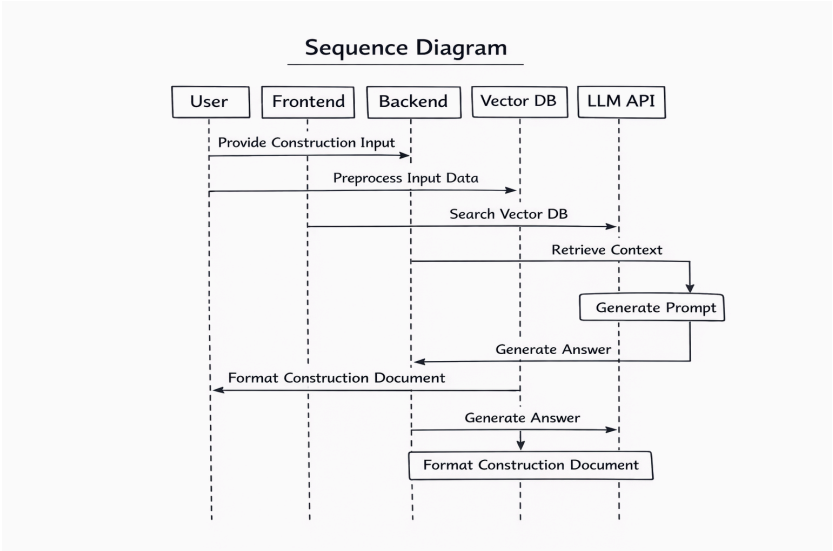
- Raw input is converted into a Python dictionary.
- Cleaned input is passed to the Query Builder.
- Query Builder produces a search query string.
- Vector DB returns relevant text chunks.

- Prompt Builder merges retrieved text and input details.
- LLM API produces generated text.
- Output Formatter returns the final report.

Output flow

The output is a professional construction document containing:

- structured headings
- bullet points or short paragraphs
- consistent engineering-style language
- domain-specific terminology where relevant



2.4 Components Design

1. Input Handler Module

Purpose: Collect user data in structured format.

Function:get_user_input() -> dict

Output: Dictionary of report fields.

2. Preprocessing Module

Purpose: Standardize and validate input.

Responsibilities:

- remove extra spaces
- validate mandatory values
- replace blanks with "Not provided"

Function:clean_input(user_data: dict) -> dict

3. Query Builder Module

Purpose: Create semantic query for retrieval.

Function:build_search_query(user_data: dict) -> str

Example query: Daily Site Report foundation work progress safety construction template terminology

4. Vector Retrieval Module

Purpose: Retrieve construction-specific knowledge from Vector DB.

Stored knowledge includes:

- construction templates
- safety guidelines
- terminology
- documentation phrases

Function:retrieve_context(query: str, n_results: int = 3) -> str

5. Prompt Builder Module

Purpose: Construct final prompt for generation.

Function:build_prompt(user_data: dict, retrieved_context: str) -> tuple[str, str]

Returns:

- instructions
- prompt

6. LLM Integration Module

Purpose: Generate final report using LLM API.

Function:generate_document(instructions: str, prompt: str) -> str

7. Output Formatter Module

Purpose: Format and clean generated output.

Function:format_output(generated_text: str) -> str

8. Main Controller Module

Purpose: Execute end-to-end pipeline.

Function:run_generation_pipeline(user_data: dict) -> str

2.5 Key Design Considerations

1. Modularity

Each step is separated into its own module so the system remains easy to update and test.

2. Domain-specific generation

The system is designed specifically for construction documentation, not generic text generation. This improves relevance and usability.

3. Retrieval-Augmented Generation

Using Vector DB before generation improves:

- relevance
- terminology consistency
- document structure quality

4. Handling missing values

The system avoids hallucination by replacing absent values with "Not provided" rather than allowing the model to invent details.

5. Prompt control

Prompt engineering is used to enforce:

- formal engineering language
- consistent formatting
- report structure
- practical tone

6. Extensibility

New report types can be added by:

- adding more knowledge files
- creating new prompt templates
- extending document-type logic

7. Error tolerance

If retrieval fails, the system still generates output using available input and fallback instructions.

8. Maintainability

Clear module boundaries make debugging and future improvements easier.

2.6 API Catalogue

Since this project uses **Python + Vector DB + LLM API**, the main interfaces are function-level APIs inside the application and external service calls.

Internal Application APIs / Functions

1. get_user_input

get_user_input() -> dict

Description: Captures structured construction input from the user.

2. clean_input

clean_input(user_data: dict) -> dict

Description: Cleans and normalizes the input dictionary.

3. build_search_query

build_search_query(user_data: dict) -> str

Description: Builds semantic query text for Vector DB retrieval.

4. retrieve_context

retrieve_context(query: str, n_results: int = 3) -> str

Description: Returns top relevant construction knowledge from Vector DB.

5. build_prompt

build_prompt(user_data: dict, retrieved_context: str) -> tuple[str, str]

Description: Creates the prompt and instructions for the LLM.

6. generate_document

generate_document(instructions: str, prompt: str) -> str

Description: Calls the LLM API and returns generated report text.

7. format_output

format_output(generated_text: str) -> str

Description: Formats final generated content for presentation.

8. run_generation_pipeline

run_generation_pipeline(user_data: dict) -> str

Description: Controls the complete flow from input to output.

External Service APIs

1. Vector DB Query API

Used to retrieve relevant construction knowledge.

Input: semantic query embedding or query text
Output: relevant documents/chunks from stored knowledge base

2. LLM API

Used to generate the final construction report.

Input: instructions + user prompt + retrieved context
Output: generated structured construction document

3. References

1. Python official documentation
2. Vector Database documentation used in the project
3. LLM API documentation used for content generation
4. Construction report templates and standard terminology references
5. Prompt engineering concepts used for controlling structured AI output